CAMPUS CAFÉ ORDERING SYSTEM

Java OOP Laboratory Exercise

Group Member:

Adaza, Dazzle Mc Kenzie

Gella, Anjo S.

Section: E

Date: 11/10/2026

Submitted as a demonstration of classes, objects, constructors, encapsulation, inheritance, polymorphism, abstraction, and conditional statements.

. Introduction

The Campus Cafe Ordering System is a Java console program that records a customer's order of up to three menu items. It supports beverages, pastries, and meals. Each item type calculates its price using its own rules. Student customers receive a ten-percent discount, and customers earn one loyalty point for every 50 pesos spent.

1. OOP Concept Explanations

Classes and Objects: The program has seven classes: MenuItem, Beverage, Pastry, Meal, Customer, Order, and CafeApp. Objects are created in CafeApp to represent customers, food items, and orders.

Attributes and Behaviors: Each class has fields that store information and methods that perform actions. For example, Beverage stores its size and computes its price.

Object Interaction: Order stores a Customer and up to three MenuItem references. It calls item methods to compute prices and updates the customer's loyalty points after payment.

Abstraction: MenuItem is an abstract class with the abstract computePrice() method. Each concrete menu-item subclass provides its own implementation.

Inheritance: Beverage, Pastry, and Meal extend MenuItem, so they reuse its common name, base price,item code, and methods.

Encapsulation: Fields are private or protected and are accessed through getters and setters. Setters validate values such as negative prices and negative extra-side counts.

Polymorphism and Overriding: Order stores different item types using MenuItem references. Calls to computePrice() and getDescription() use the subclass implementations at runtime.

Method Overloading: Customer has addPoints(int) and addPoints(double). Order has computeTotal()and computeTotal(double), allowing different parameter lists.

2. **Program Output**

The following is the actual console output produced after compiling and running the submitted Java files:

```
Order is full. Cannot add Cookie.
Error: base price cannot be negative.


\nOrder #1001 — Dazzle Mc Kenzie Adaza(Student)
Iced Latte [B-01] Size: L k                    120.00
Ensaymada [P-01] (Warmed)                       55.00
Chicken Adobo [M-01] Upsized, 1 side           142.00
Subtotal: 317.00
Student discount (10%): -31.70
TOTAL:    285.30
Loyalty points: 5
Error: base price cannot be negative.


\nOrder #1002 — Anjo Gella
Hot Chocolate [B-02] Size: M                    95.00
Beef Tapa [M-02] Regular, 0 sides              130.00
Subtotal: 225.00
Voucher: -20.00
TOTAL: 205.00
Loyalty points: 4
```

3**. Challenges Encountered and Solutions**

One challenge was designing the Order class so it could store different menu-item types without using arrays or collection classes. This was solved by creating three separate MenuItem fields and checking each field before adding an item. Another challenge was making sure each item had its own pricing rule.This was solved by declaring computePrice() as an abstract method in MenuItem and overriding it in Beverage, Pastry, and Meal. Input validation was handled through setters so invalid negative values are rejected while valid data is preserved.

4**. Program features**

- Supports up to three menu items per order.
- Calculates beverage prices for small, medium, and large sizes.
- Adds a warming fee to pastries when selected.
- Adds upsizing and extra-side fees to meals.
- Applies a ten-percent student discount.
- Supports voucher deductions without allowing a negative total.
- Awards one loyalty point for every 50 pesos spent.
- Prints receipts and demonstrates rejection of a fourth item and invalid negative price

5**. Testing**

The CafeApp main method creates two customers and demonstrates beverages, pastries, and meals. It prints two receipts, tests the three-item limit by attempting to add a fourth item, applies a voucher to the second order, and attempts to set a negative base price to demonstrate validation.

6. **Conclusion**

The project demonstrates how Java OOP concepts can be combined to create a small ordering system. The use of abstraction, inheritance, encapsulation, polymorphism, constructors, and conditional statements makes the program organized and easier to maintain.